



Universidad de Palermo

Facultad de Ingeniería

Materia Electiva

Microservicios y APIs Escalables

Cronograma de Cursada y Plan de Trabajo

Ing. Jorge Agustín Pereyra · 2026

1. Información General de la Materia

Campo	Detalle
Institución	Universidad de Palermo
Materia	Microservicios y APIs Escalables
Tipo	Materia Electiva
Docente	Ing. Jorge Agustín Pereyra
Año	2026
Total de clases	16 (13 funcionales + 1 parcial + 1 oral + 1 recuperatorio)
Duración de clase	3 horas
Modalidad de trabajo	Parejas (TP integrador) — Evaluaciones individuales
Repositorio	GitHub (una rama por sprint)

2. Descripción de la Materia

Microservicios y APIs Escalables es una materia electiva orientada a estudiantes avanzados de Ingeniería en Sistemas que ya conocen los fundamentos del desarrollo web con Java y Spring Boot.

El objetivo central es que cada estudiante, al finalizar la cursada, haya construido y desplegado un sistema real de software —una clínica veterinaria— que evoluciona sprint a sprint desde una arquitectura monolítica hasta una arquitectura de microservicios completa, aplicando las mismas tecnologías y prácticas que utiliza la industria hoy.

3. Roles en el Equipo de Trabajo

Durante la cursada simulamos un equipo real de desarrollo de software con dos roles bien diferenciados:

3.1 Ingeniero de Software (el docente)

- Define la arquitectura del sistema y toma las decisiones técnicas de cada sprint
- Actúa como Product Owner: redacta el backlog y los criterios de aceptación
- Realiza el live coding de la parte arquitectural clave (aprox. 30 minutos por clase)
- Explica el POR QUÉ de cada patrón, tecnología y decisión de diseño
- Audita el código entregado y conduce la retrospectiva

3.2 Programador Junior (los estudiantes)

- Completa la funcionalidad que no se cubrió en el live coding
- Implementa endpoints, tests y validaciones del sprint
- Realiza code review con su compañero de pareja antes de entregar
- Justifica sus decisiones técnicas en la retrospectiva y en la evaluación oral
- Cumple con los criterios de aceptación del sprint (Definition of Done)

Aclaración importante

El live coding NO es un tutorial paso a paso que se copia.

El ISW muestra la estructura y el patrón arquitectural. La pareja resuelve el resto.

En la evaluación oral, cada estudiante debe poder defender cualquier línea de su código.

4. Metodología de Trabajo

4.1 ¿Por qué metodología ágil?

Trabajamos con una metodología ágil inspirada en Scrum porque es el estándar actual de la industria de software. Cada clase equivale a un sprint con entregable concreto.

Waterfall	Ágil (esta materia)
El resultado se ve al final del cuatrimestre	Cada sprint tiene un entregable funcional

Waterfall	Ágil (esta materia)
Un error tarde implica rehacer todo	Los errores se detectan y corrigen sprint a sprint
Sin feedback hasta la entrega final	Retrospectiva al cierre de cada clase
No refleja cómo trabaja la industria	Es exactamente cómo trabaja la industria

4.2 Estructura de cada clase (sprint)

Momento	Tiempo	Responsable	Descripción
01 · Revisión sprint anterior	15 min	Parejas	Demo rápida de lo entregado. El ISW da feedback.
02 · Teoría + Arquitectura	20 min	ISW	Concepto del sprint. Decisión de diseño. ¿Por qué?
03 · Live Coding	30 min	ISW	Codea la parte arquitectural clave. Las parejas observan y preguntan.
04 · Sprint de las parejas	40 min	Estudiantes	Completan el backlog del sprint. ISW rota y asesora.
05 · Retrospectiva / DoD	15 min	Todos	Verifican Definition of Done. ¿Qué aprendimos?

4.3 Definition of Done (DoD)

No alcanza con que el sistema 'más o menos ande'. Cada sprint tiene criterios de aceptación concretos:

- El código compila y todos los tests pasan
- El o los endpoints responden correctamente (verificado con Postman o similar)
- El código está commiteado en la rama del sprint en el repositorio del grupo
- El README del sprint tiene instrucciones de ejecución actualizadas
- La pareja puede hacer demo en vivo sin errores críticos

4.4 Evaluación

Instancia	Modalidad	Peso	¿Qué se evalúa?
Parcial 1	Individual	40%	Teoría (MVC, patrones, REST, JPA) + API veterinaria funcionando

Instancia	Modalidad	Peso	¿Qué se evalúa?
TP Final — Oral	Individual	40%	Demo en vivo del sistema completo + defensa de decisiones de arquitectura
Sprints / Participación	Por pareja	20%	Entregas parciales, code review, actitud en el equipo ágil

Notas sobre la evaluación

El recuperatorio cubre el Parcial 1 y/o la re-entrega oral para quienes no alcanzaron la nota mínima.

En la oral se espera que cada estudiante defienda al menos UNA decisión de arquitectura difícil.

La nota de participación se construye sprint a sprint — no se puede recuperar al final.

5. El Proyecto Integrador: Clínica Veterinaria

A lo largo de toda la cursada las parejas construyen el sistema de gestión de una clínica veterinaria. El dominio es deliberadamente simple para que el foco esté en la arquitectura, no en el negocio.

5.1 Entidades del dominio

Entidad	Atributos principales	Relaciones
Dueño	nombre, apellido, dni, teléfono, email	Tiene muchas Mascotas
Mascota	nombre, especie, raza, fechaNacimiento	Pertenece a un Dueño · tiene muchos Turnos
Veterinario	nombre, apellido, matrícula, especialidad	Atiende muchos Turnos
Turno	fecha, hora, motivo, estado, observaciones	Tiene una Mascota y un Veterinario

5.2 Stack tecnológico completo

Tecnología	Versión	Propósito	Fase
Spring Boot	4.1.0	Backend principal de todos los servicios	1 y 2
Bootstrap + JS	5.3 / Vanilla	Frontend web — UI de la clínica veterinaria	1 y 2
JPA / Hibernate	via Spring	Persistencia relacional	1
MySQL	8.x	Base de datos relacional	1 y 2
MongoDB	7.x	Historial de turnos (documento)	2
Redis	7.x	Caché de veterinarios disponibles	2
Git / GitHub	—	Control de versiones y entrega por sprint	1 y 2
Eureka	Spring Cloud	Service Registry y Discovery	2
Spring Cloud Config	Spring Cloud	Configuración centralizada vía Git	2
Spring Cloud Gateway	Spring Cloud	API Gateway + Resilience4J	2
OpenFeign	Spring Cloud	Comunicación síncrona entre microservicios	2
Spring Security + JWT	6.x	Autenticación y autorización cross-servicios	2
Actuator + Micrometer	via Spring	Health checks y métricas de cada servicio	2
Docker / Compose	—	Orquestación del stack completo	2
Swagger / OpenAPI	3.x	Documentación de APIs	1
JUnit 5 + Mockito	via Spring	Testing unitario e integración	1 y 2

5.3 Microservicios que construyen

Microservicio	Puerto	Base de datos	Responsabilidad
eureka-server	8761	—	Service Registry
config-server	8888	Git repo	Configuración centralizada
api-gateway	8080	—	Ruteo + Resilience4J + JWT
ms-dueno	8081	MySQL	CRUD de dueños

Microservicio	Puerto	Base de datos	Responsabilidad
ms-mascota	8082	MySQL	CRUD de mascotas
ms-veterinario	8083	MySQL + Redis	CRUD de veterinarios + caché
ms-turno	8084	MySQL + MongoDB	Gestión de turnos + historial

6. Cronograma de Cursada

FASE 1 — Monolito MVC (Sprints 1 a 7)

Sprint 1 · Clase	
Tema	Kickoff + dominio + setup del proyecto
Práctica	Modelado de entidades en papel · Proyecto Spring Boot 4.1.0 · Repositorio GitHub
Stack	Spring Boot 4.1.0 · Git · GitHub

Sprint 2 · Clase	
Tema	Arquitectura MVC + REST
Práctica	CRUD completo de Dueño · Controller → Service → Repository · Postman
Stack	Spring MVC · REST · @RestController

Sprint 3 · Clase	
Tema	Persistencia JPA + relaciones
Práctica	CRUD Mascota · @OneToMany @ManyToOne · MySQL local
Stack	JPA · Hibernate · MySQL 8

Sprint 4 · Clase

Tema	CRUD Turno + Veterinario + DTOs
Práctica	CRUD Turno y Veterinario · DTOs con MapStruct · Lombok
Stack	MapStruct · Lombok · JPA

Sprint 5 · Clase

Tema	Validaciones + manejo de errores
Práctica	Bean Validation en todos los endpoints · @ControllerAdvice global · Respuestas HTTP consistentes
Stack	Bean Validation · @ControllerAdvice · HTTP status codes

Sprint 6 · Clase

Tema	Testing: JUnit 5 + Mockito
Práctica	Tests unitarios de Services · MockMvc para Controllers · Arrange-Act-Assert
Stack	JUnit 5 · Mockito · MockMvc

Sprint 7 · Clase

Tema	Swagger + Bootstrap UI + preparación migración
Práctica	OpenAPI docs · UI Bootstrap 5 + JS Fetch · Análisis de pain points del monolito
Stack	Swagger/OpenAPI 3 · Bootstrap 5 · JS Vanilla



PARCIAL 1 — Evaluación individual · Teoría + API veterinaria funcionando

FASE 2 — Microservicios (Sprints 9 a 14)

Sprint 9 · Clase

Tema	¿Por qué microservicios? + Arquitectura Hexagonal
Práctica	Análisis de bounded contexts · Estructura hexagonal en ms-dueno (domain / application / infrastructure)
Stack	Arquitectura Hexagonal · Bounded Context · DDD básico

Sprint 10 · Clase

Tema	Eureka + Config Server + Actuator
Práctica	Eureka Server · Config Server con Git repo · /actuator/health en cada servicio · Dashboard Eureka
Stack	Spring Cloud Netflix Eureka · Spring Cloud Config · Actuator · Micrometer

Sprint 11 · Clase

Tema	API Gateway + Resilience4J (Circuit Breaker)
Práctica	Gateway con rutas a cada MS · Filtros · Circuit Breaker en ruta ms-veterinario · Fallback response
Stack	Spring Cloud Gateway · Resilience4J · Circuit Breaker

Sprint 12 · Clase

Tema	Spring Security + JWT
Práctica	Login básico · Roles ADMIN y VETERINARIO · El Gateway valida el token antes de rutear
Stack	Spring Security 6 · JWT · Bearer Token

Sprint 13 · Clase

Tema	OpenFeign — comunicación entre microservicios
Práctica	ms-turno llama a ms-mascota y ms-veterinario via Feign · Manejo de errores cross-service
Stack	OpenFeign · Spring Cloud LoadBalancer

Sprint 14 · Clase	
Tema	Redis + MongoDB + Docker Compose
Práctica	Caché Redis en ms-veterinario · Historial de turnos en MongoDB · docker-compose levanta el stack completo
Stack	Redis 7 · MongoDB 7 · Docker Compose



PRESENTACIÓN ORAL — Individual · Demo en vivo del sistema completo + defensa de arquitectura



RECUPERATORIO — Individual · Parcial 1 y/o re-entrega oral

7. Organización del Repositorio

Aspecto	Detalle
Plataforma	GitHub — un repositorio privado por pareja con acceso al docente
Ramas	Una rama por sprint: sprint-01, sprint-02, ..., sprint-14
Entrega	Cada sprint se entrega como Pull Request a main con descripción de lo implementado
README	Cada PR debe incluir instrucciones de ejecución actualizadas
Convención de commits	feat: / fix: / refactor: / test: + descripción en español